For Google Colab

1. CSV File

```
%%writefile sports_data.csv
Player Name,Team Name,Match Played,Runs,Wickets
Virat,India,5,420,2
Rohit,India,5,350,1
Kane,New Zealand,5,390,3
Glen,New Zealand,5,100,10
Smith,Australia,5,370,2
Starc,Australia,5,120,9
```

2. Sports_module

```
%%writefile sports_module.py
def calculate_performance(runs, wickets):
    score = runs + (wickets * 10)
    return score
def find_best_player(data):
    best_player = ""
    best_score = 0
    for player in data:
        score = calculate_performance(
            player["Runs"],
            player["Wickets"]
        )
        if score > best_score:
            best_score = score
            best_player = player["Player Name"]
    return best_player, best_score
def find_best_team(team_data):
    best_team = ""
    highest_runs = 0
    for team in team_data:
        if team_data[team]["Runs"] > highest_runs:
            highest_runs = team_data[team]["Runs"]
            best_team = team
    return best_team, highest_runs
```

3. Main Program

```
# ==========================================
# SPORTS ANALYTICS USING PYTHON
# ==========================================

# C. Python Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# J. SciPy
from scipy.stats import pearsonr

# D. Custom Module
import sports_module



# ==========================================
# A. PYTHON CORE CONCEPTS
# ==========================================

print("=" * 50)
print("       SPORTS ANALYTICS USING PYTHON")
print("=" * 50)

project_name = "Sports Analytics"
number_of_players = 6

print("Project:", project_name)
print("Number of Players:", number_of_players)



# ==========================================
# G. FILE HANDLING + E. EXCEPTION HANDLING
# ==========================================

try:
```

```
    # K. Pandas reads the separate CSV file
    df = pd.read_csv("sports_data.csv")

    print("\nCSV file loaded successfully!")

except FileNotFoundError:

    print("\nError: sports_data.csv file not found.")
    exit()

except Exception as e:

    print("\nError:", e)
    exit()


# ==========================================
# DISPLAY DATA
# ==========================================

print("\n--- SPORTS DATASET ---")
print(df)


# ==========================================
# B. DATA STRUCTURES
# ==========================================

# List
player_names = df["Player Name"].tolist()

print("\n--- PLAYER LIST ---")
print(player_names)


# Dictionary
```

```
player_data = {
    "Player": "Virat",
    "Team": "India",
    "Runs": 420,
    "Wickets": 2
}

print("\n--- PLAYER DICTIONARY ---")
print(player_data)


# ==========================================
# A. FUNCTIONS AND LOOPS
# ==========================================

def calculate_average(runs):

    total = 0

    for value in runs:
        total = total + value

    average = total / len(runs)

    return average


average_runs = calculate_average(
    df["Runs"].tolist()
)

print("\nAverage Runs:", round(average_runs, 2))


# ==========================================
# I. NUMPY
# ==========================================
```

```
runs_array = np.array(df["Runs"])

print("\n--- NUMPY ANALYSIS ---")

print("Total Runs:", np.sum(runs_array))
print("Highest Runs:", np.max(runs_array))
print("Lowest Runs:", np.min(runs_array))
print("Average Runs:", round(np.mean(runs_array), 2))



# ==========================================
# PLAYER PERFORMANCE
# ==========================================

df["Performance Score"] = (
    df["Runs"] + (df["Wickets"] * 10)
)

print("\n--- PLAYER PERFORMANCE ---")

print(
    df[
        [
            "Player Name",
            "Team Name",
            "Runs",
            "Wickets",
            "Performance Score"
        ]
    ]
)



# ==========================================
# CUSTOM MODULE
# ==========================================
```

```
# Convert DataFrame into list of dictionaries
data = df.to_dict("records")

best_player, best_score = sports_module.find_best_player(data)

print("\n--- BEST PLAYER ---")
print("Best Performing Player:", best_player)
print("Performance Score:", best_score)


# ==========================================
# TEAM PERFORMANCE USING PANDAS
# ==========================================

team_data = df.groupby("Team Name").agg({
    "Runs": "sum",
    "Wickets": "sum"
})

print("\n--- TEAM PERFORMANCE ---")
print(team_data)


# ==========================================
# CONVERT TEAM DATA TO DICTIONARY
# ==========================================

team_dictionary = {}

for team in team_data.index:

    team_dictionary[team] = {
        "Runs": team_data.loc[team, "Runs"],
        "Wickets": team_data.loc[team, "Wickets"]
    }
```

```
# Find best team using custom module
best_team, highest_team_runs = sports_module.find_best_team(
    team_dictionary
)

print("\nBest Team:", best_team)
print("Total Runs:", highest_team_runs)


# ==========================================
# J. SCIPY STATISTICAL ANALYSIS
# ==========================================

runs = df["Runs"].to_numpy()
wickets = df["Wickets"].to_numpy()

correlation, p_value = pearsonr(runs, wickets)

print("\n--- SCIPY ANALYSIS ---")

print(
    "Correlation between Runs and Wickets:",
    round(correlation, 2)
)

print(
    "P-Value:",
    round(p_value, 4)
)


# ==========================================
# C. DATA VISUALIZATION
# ==========================================

plt.figure(figsize=(8, 5))
```

```
plt.bar(
    df["Player Name"],
    df["Runs"]
)

plt.xlabel("Players")
plt.ylabel("Runs")
plt.title("Player Performance Analysis")

plt.xticks(rotation=30)
plt.tight_layout()

plt.show()


# ========================================
# TEAM PERFORMANCE GRAPH
# ========================================

plt.figure(figsize=(8, 5))

plt.bar(
    team_data.index,
    team_data["Runs"]
)

plt.xlabel("Teams")
plt.ylabel("Total Runs")
plt.title("Team Performance Analysis")

plt.tight_layout()

plt.show()


# ========================================
```

```
# TOP PERFORMERS
# ==========================================

print("\n--- TOP PERFORMERS ---")

top_players = df.sort_values(
    "Performance Score",
    ascending=False
)

print(
    top_players[
        [
            "Player Name",
            "Team Name",
            "Performance Score"
        ]
    ]
)


# ==========================================
# FINAL RESULT
# ==========================================

print("\n" + "=" * 50)
print("            FINAL RESULTS")
print("=" * 50)

print("Best Performing Player :", best_player)
print("Best Team              :", best_team)
print("Average Runs           :", round(average_runs, 2))
print("Highest Runs           :", np.max(runs_array))
print("Lowest Runs            :", np.min(runs_array))
print("Correlation            :", round(correlation, 2))

print("\nSports Analytics Completed Successfully!")
```

For VS

1. Sports_data.csv

Player Name,Team Name,Match Played,Runs,Wickets
Virat,India,5,420,2
Rohit,India,5,350,1
Kane,New Zealand,5,390,3
Glen,New Zealand,5,100,10
Smith,Australia,5,370,2
Starc,Australia,5,120,9

2. Sports_module.py

```
# sports_module.py
def calculate_performance(runs, wickets):
    """Calculate player performance score."""
    score = runs + (wickets * 10)
    return score
def find_best_player(data):
    """Find the best performing player."""
    best_player = ""
    best_score = 0
    for player in data:
        score = calculate_performance(
            player["Runs"],
            player["Wickets"]
        )
        if score > best_score:
            best_score = score
            best_player = player["Player Name"]
    return best_player, best_score
def find_best_team(team_data):
    """Find the team with highest total runs."""
    best_team = ""
    highest_runs = 0
    for team in team_data:
        if team_data[team]["Runs"] > highest_runs:
            highest_runs = team_data[team]["Runs"]
```

```
        best_team = team
    return best_team, highest_runs
```

3. Main Program
Same as google colab

Covered Topics in code:

Poster topic/content	Covered in code?	Where it is used
A. Python Core Concepts	✅ Yes	Variables, data types, operators, loops, functions
B. Data Structures	✅ Yes	Lists and Dictionaries
C. Python Libraries	✅ Yes	Pandas, NumPy, Matplotlib, SciPy
D. Custom Modules	✅ Yes	`sports_module.py`
E. Exception Handling	✅ Yes	`try-except` for CSV file
G. File Handling	✅ Yes	Reading `sports_data.csv` using Pandas
I. NumPy	✅ Yes	`np.array()`, `np.sum()`, `np.max()`, `np.min()`, `np.mean()`
J. SciPy	✅ Yes	`pearsonr()` for correlation
K. Pandas	✅ Yes	`read_csv()`, DataFrame, `groupby()`, `sort_values()`, `to_numpy()`